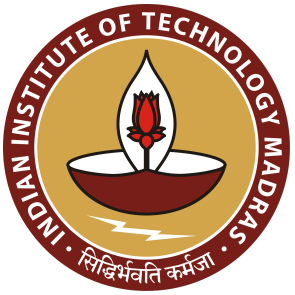
GRAND CHALLENGE - 2025
IP Integration with Shakti SoC



DEVELOPED BY :
SHAKTI DEVELOPMENT TEAM @ IITM
SHAKTI.ORG.IN

0.1 Proprietary Notice **3**
0.2 Release Information **4**
Introduction **5**
Integrating existing Slow peripherals with Shakti SoC **5**
Integrating existing Fast peripherals with Shakti SoC **10**
Passing Interrupt to PLIC **13**

0.1 Proprietary Notice

Copyright © 2025, **SHAKTI@IIT Madras**.All rights reserved.
Information in this document is provided “as is,” with every effort ensuring that the documentation is as accurate as possible.

SHAKTI@IIT Madras expressly disclaims all warranties, representations, and conditions of any kind, whether express or implied, including, but not limited to, the implied warranties or conditions of merchantability, fitness for a particular purpose and non-infringement.

SHAKTI@IIT Madras does not assume any liability rising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation indirect, incidental, special, exemplary, or consequential damages.

SHAKTI@IIT Madras reserves the right to make changes without further notice to any products herein.

The project was funded by Ministry of Electronics and Information Technology (MeitY), Government of India

0.2 Release Information

Version	Date	Updates	Reviewed by
1.0	**16/09/2025**	**Poovarasan & Gayathri**	**Kotteeswaran E**
Draft V.1	**07/10/2025**	**Poovarasan M**	**Kotteeswaran E**

Introduction

The peripherals being used in a SoC can be configured based on the end application. Few applications need new IPs that need to be added to SoC.

The above said integration can be classified into the following two variants,

- Integrating existing Slow peripherals with Shakti SoC.
- Integrating existing Fast peripherals with Shakti SoC.

Note: Here the existing peripherals means that the IPs developed inhouse by Shakti Team.

The files specified in this document are present in the gc2025/hw/boards/arty_a7_yamuna/ directory. Compare the images with the original content of files to observe the changes, most of the changes will be newly added lines, only some lines will be modified. The same steps will be applicable to the integration of custom Verilog IP with Ganga SoC also, but the changes alone should be made in gc2025/hw/boards/arty_a7_ganga/ directory.

To integrate the existing peripheral / IP which operates at higher frequency AXI4 interface [Fast Fabric] will be used, whereas for existing peripheral / IP which operates at lower frequency AXI4 Lite interface [Slow Fabric] can be used.

This document details the procedure that needs to be followed when integrating those IPs into Shakti SoC.

Note: The highlighted lines in the following images will indicate the changes. The line numbers shown in the images might not match exactly.

Integrating existing Slow peripherals with Shakti SoC

Slow peripherals operate at lower frequencies from few Hz to few MHz (upto 25MHz). They can be integrated with Shakti core using the AXI4 Lite interface which is slow fabric.

The peripherals like SPI, UART, I2C, GPIO, etc. are interfaced with Shakti using the following logic. In addition, it is assumed that the peripheral IP is already integrated with Shakti. The objective is to add one more additional instantiation of the same peripheral.

Normally these peripherals are interfaced with Core through clusters (e.g. UART cluster, SPI cluster, PWM cluster, Mixed cluster, etc.). This section explains how to add another instantiation of an UART to the Shakti Core.

File: “gc2025/hw/boards/arty_a7_yamuna/Soc.defines”

Step 1: Extend the Memory range of UART cluster to accommodate 1 more UART

```
68   `define UARTClusterBase    'h0001_1300
69   `define UARTClusterEnd     'h0001_1640
70   `define SPIClusterBase     'h0002_0000
71   `define SPIClusterEnd      'h0002_01FF
```

Step 2: Increment the “**UARTCluster_Num_Slaves**” by 1

```
104  // - UART cluster
105  `define UARTCluster_Num_Slaves 5
106  `define UARTCluster_Num_Masters 1
107
```

Step 3: Add “**`define UART3_slave_num *n***” (replace the *n* by the last slave number - 1) and increment the UARTCluster_err_slave_num number by 1.

```
108  `define UART0_slave_num 0
109  `define UART1_slave_num 1
110  `define UART2_slave_num 2
111  `define UART3_slave_num 3
112  `define UARTCluster_err_slave_num 4
113
```

Step 4: Add “**`define UART3Base *addr1**” and “**`define UART3End *addr2**” . Define *addr1 and *addr2 such that it does not conflict with other slaves of the UART cluster.

```
114  `define UART0Base      'h0001_1300
115  `define UART0End       'h0001_1340
116  `define UART1Base      'h0001_1400
117  `define UART1End       'h0001_1440
118  `define UART2Base      'h0001_1500
119  `define UART2End       'h0001_1540
120  `define UART3Base      'h0001_1600
121  `define UART3End       'h0001_1640
```

File: “gc2025/hw/boards/arty_a7_yamuna/uart_cluster.bsv”

Step 1: Add the decoding logic for “**UART3_slave_num**” selection.

```
41  function Bit#(TLog#(`UARTCluster_Num_Slaves)) fn_slave_map (Bit#(`paddr) addr);
42    Bool slave_exist = True;
43    Bit#(TLog#(`UARTCluster_Num_Slaves)) slave_num = 0;
44    if(     addr>= `UART0Base && addr<= `UART0End )
45      slave_num =  `UART0_slave_num;
46    else if(addr>= `UART1Base && addr<= `UART1End )
47      slave_num =  `UART1_slave_num;
48    else if(addr>= `UART2Base && addr<= `UART2End )
49      slave_num =  `UART2_slave_num;
50    else if(addr>= `UART3Base && addr<= `UART3End )
51      slave_num = `UART3_slave_num;      You, 14 seconds ago • Uncommitted change
52    else
53      slave_num = `UARTCluster_err_slave_num;
54
55    return slave_num;
56  endfunction:fn_slave_map
```

Step 2: Declare the “**RS232**” interface inside the “**Ifc_uart_cluster**” interface.

```
58  interface Ifc_uart_cluster;
59    interface RS232 uart0_io;
60    interface RS232 uart1_io;
61    interface RS232 uart2_io;
62    interface RS232 uart3_io;
63    interface AXI4_Lite_Slave_IFC#(`paddr, 32, 0) slave;
64    method Bit#(3) uart_interrupts;
65  endinterface
```

Step 3: Instantiate the another UART inside the “**mkuart_cluster**” module

```
88    let uart0 <- mkuart();
89    let uart1 <- mkuart();
90    let uart2 <- mkuart();
91    let uart3 <- mkuart();
```

Step 4: Route the declared RS232 interface with name “**uart3_io**” to the “**io**” interface of module “**uart3**”.

```
108   interface uart0_io=uart0.io;
109   interface uart1_io=uart1.io;
110   interface uart2_io=uart2.io;
111   interface uart3_io=uart3.io;
```

Step 5: Make a connection between the AXI4 lite slave interface of "**uart3**" and the AXI4 lite master interface of Shakti SoC.

```
103    mkConnection (fabric.v_to_slaves [`UART0_slave_num ],uart0.slave);
104    mkConnection (fabric.v_to_slaves [`UART1_slave_num ],uart1.slave);
105    mkConnection (fabric.v_to_slaves [`UART2_slave_num ],uart2.slave);
106    mkConnection (fabric.v_to_slaves [`UART3_slave_num ],uart3.slave);
```

File: "gc2025/hw/boards/arty_a7_yamuna/Soc.bsv"

Step 1: Declare the "**RS232**" interface inside the "**Ifc_chip_io**" interface.

```
120    interface Ifc_chip_io;
121      interface Ifc_sspi_io spi0_io;
122    //  interface Ifc_sspi_io spi1_io;
123      interface RS232 uart0_io;
124      interface RS232 uart3_io;
125      method I2C_out i2c0_out;
126      method I2C_out i2c1_out;
```

Step 2: Route the declared RS232 interface with name "**uart3_io**" to the "**uart3_io**" interface of module "**uart_cluster**".

```
658    interface uart0_io = uart_cluster.uart0_io;
659    interface uart3_io = uart_cluster.uart3_io;
660    method  i2c0_out = mixed_cluster.i2c0_out;
661    method  i2c1_out = mixed_cluster.i2c1_out;
```

File: "gc2025/hw/boards/arty_a7_yamuna/fpga_top.v"

Step 1: Declare the IO ports for the additionally added UART module.

```
64     // ---- UART ports --------//
65      input          uart0_SIN,
66      output         uart0_SOUT,
67
68      input          uart3_SIN,
69      output         uart3_SOUT,
70
```

Step 2: Connect the declared UART IO ports to the UART module instantiated in the Shakti SoC.

```
587     // UART port definitions
588     .uart0_io_SIN(uart0_SIN),
589     .uart0_io_SOUT(uart0_SOUT),
590     .uart3_io_SIN(uart3_SIN),
591     .uart3_io_SOUT(uart3_SOUT),
```

File: “gc2025/hw/boards/arty_a7_yamuna/constraints.xdc”

Step 1: Map the UART IO ports to the available pins. Find the original constraints for [Arty A7 100t.]

```
8   ## Pmod Header JB
9   set_property -dict { PACKAGE_PIN E15   IOSTANDARD LVCMOS33 } [get_ports { i2c0_sda }]; #IO_L11P_T1_SRCC_15 Sch=jb_p[1]
10  set_property -dict { PACKAGE_PIN E16   IOSTANDARD LVCMOS33 } [get_ports { i2c0_scl }]; #IO_L11N_T1_SRCC_15 Sch=jb_n[1]
11  set_property -dict { PACKAGE_PIN D15   IOSTANDARD LVCMOS33 } [get_ports { gpio_14 }]; #IO_L12P_T1_MRCC_15 Sch=jb_p[2]
12  set_property PULLDOWN true [get_ports { gpio_14 }];
13  set_property -dict { PACKAGE_PIN C15   IOSTANDARD LVCMOS33 } [get_ports { gpio_15 }]; #IO_L12N_T1_MRCC_15 Sch=jb_n[2]
14  set_property PULLDOWN true [get_ports { gpio_15 }];
15  set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { uart3_SOUT }]; #IO_L23P_T3_FOE_B_15 Sch=jb_p[3]
16  set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { uart3_SIN }]; #IO_L23N_T3_FWE_B_15 Sch=jb_n[3]
17  set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { external_clk }]; #IO_L24N_T3_RS0_15 Sch=jb_n[4]
```

Integrating existing Fast peripherals with Shakti SoC

Peripherals that operate at higher frequencies can be integrated with Shakti core using the AXI4 interface which is fast fabric. AES is taken as an example in this document.

AES tends to operate in a high frequency mostly in equivalence with the core frequency. So AES will be integrated with the SoC using AXI interface

The AES is integrated through AXI4 fast fabric, so the memory allocation for AES doesn't follow any hierarchy like there is cluster range present in the UART. The memory mapping for AES should be unique in that it doesn't fall in any of the range that is already in usage.

First clone **crypto-box** repository for AES,

$ cd gc2025/hw/
$ git clone https://gitlab.com/shaktiproject/cores/crypto-box
$ git checkout 26-32-bit-axi-datawidth #Skip this step for Ganga SoC

File: "gc2025/hw/bsvpath"

Step 1: Add path for the AES

```
6   c-class/src/mbox
7   crypto-box/aes_buffer
8   csrbox_bsv
9   fabrics/axi4
```

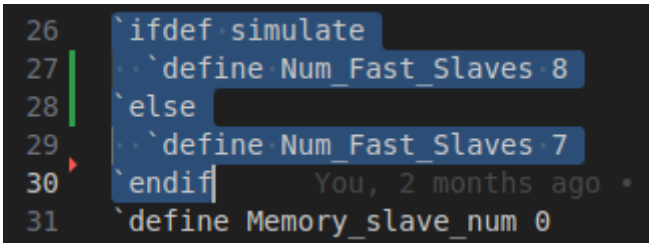
File: "gc2025/hw/Makefile"

Step 1: Copy the Block RAM and Gated Clock verilog files to the verilog directory

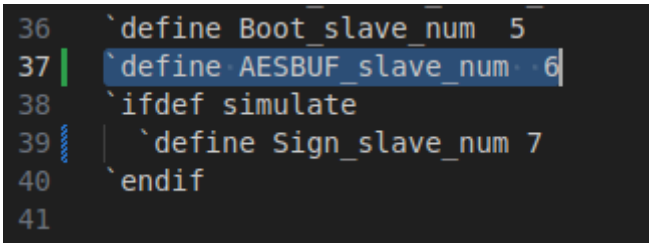
```
181  .PHONY: generate_verilog
182  generate_verilog: $(BSVBUILDDIR)/$(TOP_BIN)
183      @cp ${BS_VERILOG_LIB}/../Verilog.Vivado/RegFile.v ${VERILOGDIR}
184      @cp ${BS_VERILOG_LIB}/../Verilog.Vivado/BRAM2BELoad.v ${VERILOGDIR}
185      @cp ${BS_VERILOG_LIB}/BRAM1.v ${VERILOGDIR}
186      @cp ${BS_VERILOG_LIB}/BRAM2.v ${VERILOGDIR}
187      @cp ${BS_VERILOG_LIB}/GatedClock.v ${VERILOGDIR}
188      @cp ${BS_VERILOG_LIB}/FIFO2.v ${VERILOGDIR}
189      @cp ${BS_VERILOG_LIB}/FIFO1.v ${VERILOGDIR}
```

File: “gc2025/hw/boards/arty_a7_yamuna/Soc.defines”

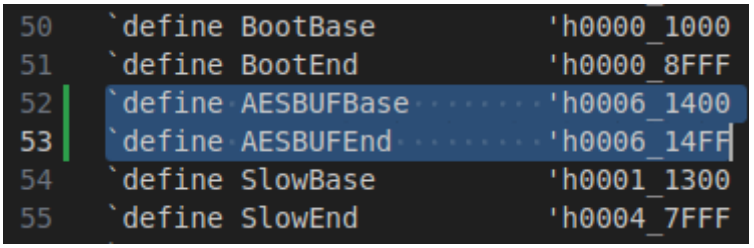
Step 1: Increment the “**Num_Fast_Slaves**” by 1



Step 2: Add “**`define AESBUF_slave_num *n***” (replace the *n* by the last slave number - 1) and increment the Sign_slave_num number by 1.

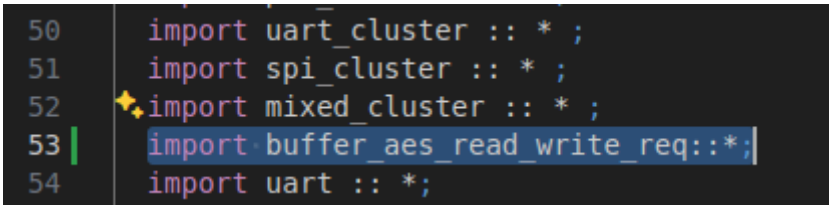


Step 3: Add “**`define AESBUFBase *addr1**” and “**`define AESBUFEnd *addr2**” . Define *addr1 and *addr2 such that it does not conflict with other slave addresses.



File: “gc2025/hw/boards/arty_a7_yamuna/Soc.bsv”

Step 1: Import the AES package “**buffer_aes_read_write_req**” in Soc



Step 2: Add the decoding logic for "**AESBUF_slave_num**" selection.

```
85      else if(addr >= `BootBase && addr <= `BootEnd)
86        slave_num = `Boot_slave_num;
87      else if(addr >= `AESBUFBase && addr <= `AESBUFEnd)
88        slave_num = `AESBUF_slave_num;        You, 45 seconds
89      else if(addr >= `SlowBase && addr <= `SlowEnd)
90        slave_num = `Slow_fabric_slave_num;
```

Step 3: Instantiate the module "**aesbuf**". The curr_clk and curr_reset is the clock and reset which the core gets, the AES operates at the frequency at which the core operates in this case.

```
330   Ifc_mixed_cluster mixed_cluster <- mkmixed_cluster(ext_clk);
331   Ifc_aesbuf_axi4#(`paddr, XLEN, 0) aesbuf <- mkaesbuf_axi4(curr_clk, curr_reset);
332   Ifc_err_slave_axi4lite#(`paddr,32,0) err_slave <- mkerr_slave_axi4lite;
```

Step 4: Make a connection between the AXI4 slave interface of "**aesbuf**" and the AXI4 master interface of Shakti SoC.

```
418   mkConnection (fabric.v_to_slaves [`Boot_slave_num], boot.slave);
419   mkConnection (fabric.v_to_slaves [`AESBUF_slave_num], aesbuf.slave);
420
```

Passing UART Interrupt to PLIC

File: “gc2025/hw/boards/arty_a7_yamuna/uart_cluster.bsv”

Step 1: Increment the width of the “**uart_interrupts**” by 1 in method declaration.

```
63      interface AXI4_Lite_Slave_IFC#(`paddr, 32, 0) slave;
64      method Bit#(4) uart_interrupts;
65    endinterface
```

Step 2: Increment the width of the “**uart_interrupts**” by 1 in method instantiation and pass the interrupt of “**uart3**” module.

```
114     method Bit#(4) uart_interrupts;
115       return {uart3.interrupt, uart2.interrupt, uart1.interrupt, uart0.interrupt};
116       //return {uart2.interrupt, uart1.interrupt, uart0.interrupt};
117     endmethod
118   endmodule
119 endpackage
```

File: “gc2025/hw/boards/arty_a7_yamuna/mixed_cluster.bsv”

Step 1: Increment the width of the “**interrupts**” by 1 in method declaration.

```
65      interface IOCellSide pinmuxtop_iocell_side;
66      interface PeripheralSide pinmuxtop_peripheral_side;
67      (*always_ready, always_enabled*)
68      method Action interrupts(Bit#(14) inp);
69      interface AXI4_Lite_Slave_IFC#(`paddr, 32, 0) slave;
70      interface AXI4_Lite_Master_IFC#(`paddr, 32, 0) xadc_master;
71    endinterface
```

Step 2: Increment the width of the wire “**wr_external_interrupts**” by 1.

```
159   Ifc_err_slave_axi4lite#(`paddr, 32, 0 ) err_slave <- mkerr_slave_axi4lite;
160   Wire#(Bit#(14)) wr_external_interrupts <- mkDWire('d0);
161   Wire#(Bit#(2)) wr_sb_ext_interrupt <- mkDWire(0);
```

Step 3: Increment the width of the “**interrupts**” by 1 in method instantiation.

```
205     method Action interrupts(Bit#(14) inp);
206       wr_external_interrupts<= inp;
207     endmethod
```

Step 4: Increment the number of sources by 1 in the PLIC module "**mkplic**" instantiation.

```
96   (*synthesize*)
97   module mkplic(Ifc_plic_axi4lite#(`paddr, 32, 0, 36, 2, 7));
98     let ifc();
99     mkplic_axi4lite#(`PLICBase) _temp(ifc);
100    return ifc;
101  endmodule
```

Step 5: Increment the width by 1 in the "**plic_inputs**" which will be the sources for PLIC. The ordering is rearranged in the way that the existing Interrupts IDs assigned to the peripherals remain unchanged.

```
169  rule rl_connect_plic_connections;
170    let tmp <- gpio.sb_gpio_to_plic.get;
171    Bit#(16) lv_gpio_intr= truncate(pack(tmp));
172    Bit#(36) plic_inputs= {wr_external_interrupts[9], wr_external_interrupts[13:10],
       wr_external_interrupts[8:6], i2c1.isint, i2c0.isint, gptimer3.sb_interrupt, gptimer2.
       sb_interrupt, gptimer1.sb_interrupt, gptimer0.sb_interrupt, lv_gpio_intr,
       wr_external_interrupts[5:0]}; #3 2025-10-08 Not Committed Yet      You, 6 seconds ago
173    plic.sb_frm_sources(plic_inputs);
174  endrule
```